

EE675 Assignment 2

Policy Gradient Methods using REINFORCE

230026

April 24, 2026

1 Part 1: Problem Setup and Environment

The predator-prey problem is modeled on a discrete $N \times N$ grid. The predator aims to capture the prey by choosing appropriate actions at each time step.

The state is defined by the positions of the predator and the prey:

$$s = (x_p, y_p, x_{prey}, y_{prey})$$

The predator can take five actions: stay, move up, move down, move left, and move right. The prey moves randomly within the grid.

A reward of +1 is received when the predator catches the prey; otherwise, the reward is 0.

This setup naturally forms a Markov Decision Process (MDP), where the objective is to learn a policy that maximizes expected cumulative reward.

Instead of solving this problem using dynamic programming, we adopt a reinforcement learning approach where the agent learns directly from interaction.

2 Part 2: Policy Network Design

2.1 Overview

The policy $\pi_\theta(a|s)$ is represented using a neural network. The network maps the state of the environment to a probability distribution over actions.

This allows the agent to learn behavior directly from experience rather than relying on manually defined rules.

2.2 State Representation

The state is represented as:

$$s = (x_p, y_p, x_{prey}, y_{prey}) \in \mathbb{R}^4$$

2.3 Action Space

The policy outputs probabilities over 5 actions:

$$\pi_\theta(a|s) \in \mathbb{R}^5$$

2.4 Neural Network Architecture

- Input Layer: 4 neurons
- Hidden Layer: 128 neurons (ReLU)
- Output Layer: 5 neurons (Softmax)

$$h = \text{ReLU}(W_1 s + b_1)$$

$$\pi_\theta(a|s) = \text{Softmax}(W_2 h + b_2)$$

2.5 Design Rationale

The architecture is kept simple because the state space is low-dimensional. A single hidden layer is sufficient to capture the required relationships.

3 Part 3: REINFORCE Algorithm and Gradient Estimation

3.1 Objective

The goal is to maximize the expected return:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[G]$$

where G is the cumulative reward.

3.2 Policy Gradient Theorem

Using the REINFORCE algorithm, the gradient is given by:

$$\nabla J(\theta) = \mathbb{E} \left[\sum_t \nabla \log \pi_{\theta}(a_t|s_t) G_t \right]$$

where G_t is the return from time step t .

3.3 Loss Function

Instead of computing the gradient explicitly, we define the loss:

$$L(\theta) = - \sum_t \log \pi_{\theta}(a_t|s_t) G_t$$

Minimizing this loss using automatic differentiation yields the required gradient.

3.4 Monte Carlo Estimation

The expectation is approximated using sampled episodes. For each episode:

- Collect states, actions, and rewards
- Compute returns G_t
- Update parameters using gradient ascent

3.5 Variance Reduction

To stabilize learning, returns are normalized:

$$G_t = \frac{G_t - \mu}{\sigma + \epsilon}$$

This helps reduce the variance of gradient estimates.

In practice, care is taken to avoid numerical instability when the variance is very small.

4 Part 4: Optimization Methods (SGA vs Adam)

4.1 Stochastic Gradient Ascent (SGA)

In SGA, parameters are updated using:

$$\theta \leftarrow \theta + \alpha \nabla J(\theta)$$

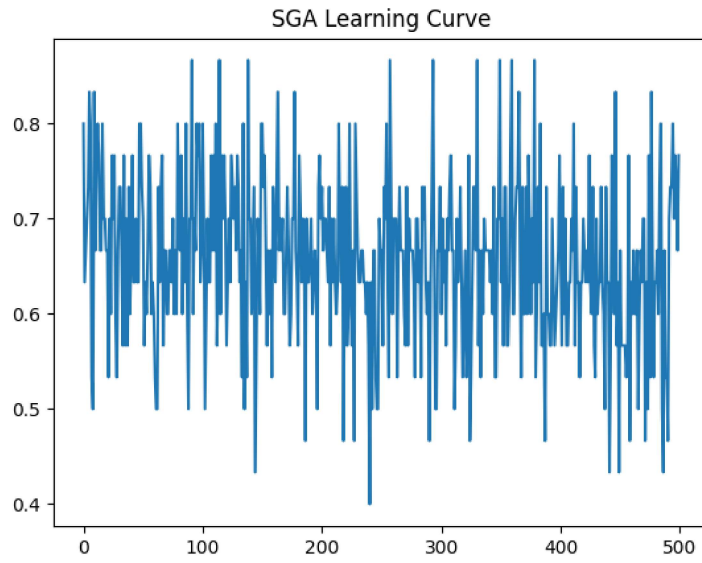


Figure 1: Learning curve using Stochastic Gradient Ascent (SGA)

where α is a fixed learning rate.

4.2 Adam Optimizer

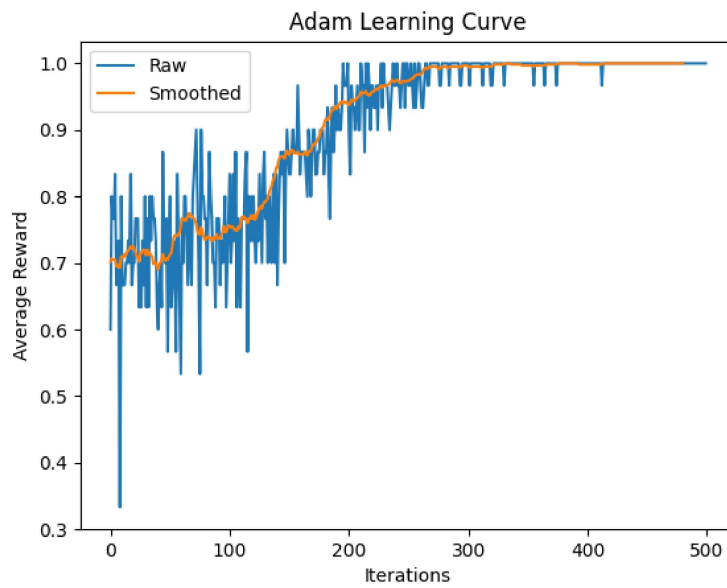


Figure 2: Learning curve using Adam Optimizer

Adam uses adaptive learning rates based on first and second moments of gradients:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

This results in more stable and efficient updates.

4.3 Comparison

- SGA exhibits high variance and unstable learning
- Adam provides smoother convergence
- Adam achieves higher final reward

Experimentally, Adam consistently outperforms SGA in both stability and convergence speed.

5 Part 5: Observations and Conclusion

5.1 Observations

- REINFORCE is simple but suffers from high variance
- SGA struggles to converge due to fixed learning rate
- Adam significantly improves training stability
- Reward curves are inherently noisy due to stochasticity

5.2 Conclusion

This assignment demonstrates the effectiveness of policy gradient methods for solving reinforcement learning problems.

While the REINFORCE algorithm provides a straightforward way to estimate gradients, its performance heavily depends on the choice of optimizer.

Adaptive methods like Adam offer substantial improvements, making them more suitable for practical applications.

Overall, the results highlight the importance of optimization techniques in reinforcement learning and show how even simple neural networks can learn effective policies.